Submission Worksheet

Submission Data

Course: IT114-007-F2025
Assignment: IT114 Milestone 2 - Hangman
Student: Evans M. (ekn3)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 11/25/2025 10:08:59 AM
Updated: 11/25/2025 11:49:24 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-2-hangman/grading/ekn3
View Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-2-hangman/view/ekn3

Instructions

1. Refer to Milestone2 of Hangman / Word guess
    1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the `Milestone2` branch
    1. `git checkout Milestone2`
    2. `git pull origin Milestone2`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone2`
    4. On Github merge the pull request from `Milestone2` to `main`
7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout main`
    2. `git pull origin main`

- Complete each section and task sequentially.
- Review the details and validation criteria for each task.
- Ensure subtasks are completed before the parent task.

Section #1: (1 pt.) Payloads

Progress: 100%

☰ Task #1 (1 pt.) - Show Payload classes and subclasses

Progress: 100%

Details:

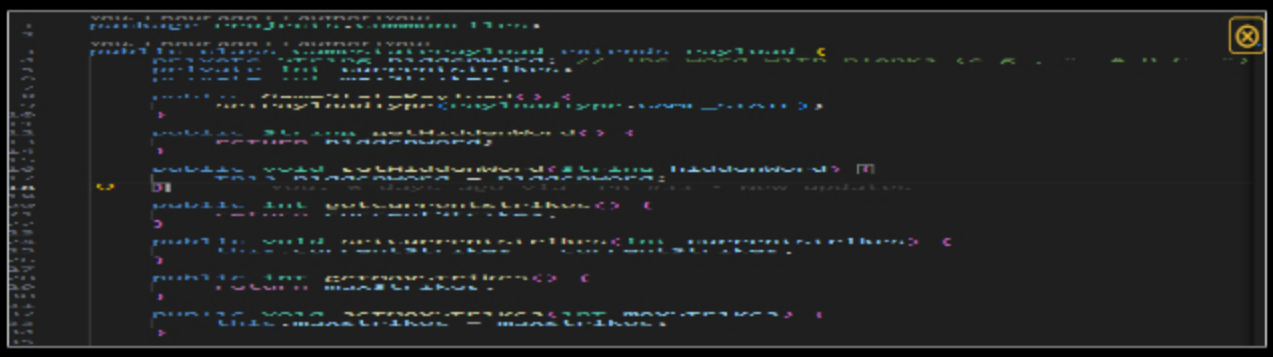
- Reqs from the document
  - Provided Payload for applicable items that only need client id, message, and type
  - PointsPayload for syncing points of players
  - Each payload will be presented by debug output (i.e, properly override the toString() method like the lesson examples)

Part 1:

Progress: 100%

Details:

- Show the code related to your payloads (Payload, PointsPayload, and any new ones added)
- Each payload should have an overriden toString() method showing its internal data



GameState Payload code



Points Payload code



GameState Payload code 2

Saved: 11/25/2025 9:42:48 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown in the screenshots and their properties

Your Response:

pointpayload updates score player which contains the player ID GameState Payload updates the word progress and strike count.

Saved: 11/25/2025 9:42:48 PM

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

Task #1 (0.57 pts.) - GameRoom Client Add/Remove

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the `onClientAdded()` code
- Show the `onClientRemoved()` code

```
/** {@inheritDoc} */
@Override
protected void onClientAdded(ServerThread sp) {
    // sync gameRoom state to new client
    sync(currentPlayer(sp));
    sync(readyStatus(sp));
    syncTurnStatus(sp);
}

/** {@inheritDoc} */
@Override
protected void onClientRemoved(ServerThread sp) {
    // stops the timers so room can clean up
    LoggerUtil.INSTANCE.info("Player Removed, remaining: " + clientsInRoom.size());
    long removedClientId = sp.getClientId();
    turnOrder.removeIf(player -> player.getClientId() == sp.getClientId());
    if (clientsInRoom.isEmpty()) {
        resetReadyTimer();
        resetTurnTimer();
        resetRoundTimer();
        onSessionEnd();
    } else if (removedClientId == currentTurnClientId) {
        onTurnStart();
    }
}
```

Code snippet for both

Saved: 11/25/2025 1:11:49 PM

Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in `onClientAdded()` (app data should at least be synchronized to the joining user)
- Briefly note the actions that happen in `onClientRemoved()` (at least should handle logic for an empty session)

Your Response:

onCleientAdded() sends the current game phase and status and also sync turns
OnClientRemoved() logs player removal, stops all timer when empty or session ends and prepare room to shut down

Saved: 11/25/2025 1:11:49 PM

Task #2 (0.57 pts.) - GameRoom Session Start

Progress: 100%

Details:

- Reqs from document
  - GameRoom loads the word list into memory from a text file for later use
  - First word is randomly chosen from in-memory list
  - Blanks are shared with all players
  - First round is triggered
- Reset/set initial state

Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionStart()`

```
/** {@inheritDoc} */
@Override
protected void onSessionStart() {
    LoggerUtil.INSTANCE.info(message: "onSessionStart() start");
    changePhase(Phase.IN_PROGRESS);      You, last week via  PR #33 • Turn based code
    currentTurnClientId = Constants.DEFAULT_CLIENT_ID;
    setTurnOrder();
    round = 0;
    LoggerUtil.INSTANCE.info(message: "onSessionStart() end");
    onRoundStart();
} <- #130-138 protected void onSessionStart()
```

on session start

Saved: 11/25/2025 9:44:40 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up initial session state for your project) and next lifecycle trigger

Your Response:

itprepares a new game start and move the game from lobby-like state into gameplay

Saved: 11/25/2025 9:44:40 PM

Task #3 (0.57 pts.) - GameRoom Round Start

Progress: 100%

Details:

- Reqs from Document
  - GameRoom round timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundStart()`

```
/** {@inheritDoc} */
@Override
protected void onRoundStart() {
    LoggerUtil.INSTANCE.info(message: "onRoundStart() start");
    resetRoundTimer();
    resetTurnStatus();
    round++;
     if (wordList.isEmpty()) {
    LoggerUtil.INSTANCE.info(message: "onRoundStart() aborted: word list is empty");
    return;
    }
    setupNewWord();
    relay(sender: null, String.format(format: "Round %d has started", round));
    LoggerUtil.INSTANCE.info(message: "onRoundStart() end");
    onTurnStart();
} <- #142-155 protected void onRoundStart()
```

code snippet

Saved: 11/25/2025 9:46:19 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the round for your project)

Your Response:

it prepares everything needed before the first player takes a turn

Saved: 11/25/2025 9:46:19 PM

Task #4 (0.57 pts.) - GameRoom Turn Start

Progress: 100%

Details:

- Reqs from Document
  - Pick next Player (Initially random, then round-robin)
  - Reset turn related status to allow player to do actions
  - GameRoom Turn timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of `onTurnStart()`

```
/** {@inheritDoc} */
@Override
protected void onTurnStart() {
    LoggerUtil.INSTANCE.info(message: "onTurnStart() start");
    resetTurnTimer();
    try {
        ServerThread currentPlayer = getNextPlayer();
        relay(sender: null, String.format(format: "It's %s's turn", currentPlayer.getDisplayName()));
    } catch (MissingCurrentPlayerException | PlayerNotFoundException e) {

        e.printStackTrace();
    }
    startTurnTimer();
    LoggerUtil.INSTANCE.info(message: "onTurnStart() end");
} <- #159-171 protected void onTurnStart()
```

onTurnStart() code snippet

Saved: 11/25/2025 9:46:50 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the turn for your project)

Your Response:

it chooses whos turn it is, notify all players and start the timer

It chooses whos turn it is, notify all players and start the timer

Saved: 11/25/2025 9:46:50 PM

Task #5 (0.57 pts.) - GameRoom Turn End

Progress: 100%

Details:

- Reqs from Document
  - **Condition 1:** Turn ends when Player guesses a letter or word (and word isn't complete)
  - **Condition 2:** Turn ends when the Turn Timer expires
  - **Condition 3:** Turn ends when the current Player Skips their turn
  - If not the last player, trigger turn start logic
  - If the last player, trigger round end logic
  - If end condition is met, trigger session end

Part 1:

Progress: 100%

Details:

- Show the snippet of `onTurnEnd()`

```
    @Override
    protected void onTurnEnd() {
        LoggerUtil.INSTANCE.info(message: "onTurnEnd() start");
        resetTurnTimer(); // reset timer if turn ended without the time expiring

        try {
            // optionally can use checkAllTookTurn();
            if (isLastPlayer()) {
                // if the current player is the last player in the turn order, end the round
                onRoundEnd();
            } else {
                onTurnStart();
            }
        } catch (MissingCurrentPlayerException | PlayerNotFoundException e) {

            e.printStackTrace();
        }
        LoggerUtil.INSTANCE.info(message: "onTurnEnd() end");
    } <- #177-194 protected void onTurnEnd()
```

on turn end code snippet

Saved: 11/25/2025 9:47:19 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the turn for your project)

Your Response:

It stops the time, decides whether to move to next turn or end the round

Saved: 11/25/2025 9:47:19 PM

Task #6 (0.57 pts.) - GameRoom Round End

Progress: 100%

Details:

- Reqs from Document
  - **Condition 1:** Round ends when word is completed
  - **Condition 2:** Round ends when Hangman Strikes reach limit
  - **Condition 3:** Round ends when the last player's turn finishes
  - If word was completed, pick new word, reset hangman, sync blanks (if not last round)
  - Send the in-progress scoreboard to all clients sorted by highest points to lowest
  - Trigger Round Start if end condition not met; otherwise, trigger session end

Part 1:

Progress: 100%

Details:

- Show the snippet of onRoundEnd()

```
/** {@inheritDoc} */
@Override
protected void onRoundEnd() {
    LoggerUtil.INSTANCE.info(message: "onRoundEnd() start");
    resetRoundTimer(); // reset timer if round ended without the time expiring

    syncScoreboard(isFinal: false);

    LoggerUtil.INSTANCE.info(message: "onRoundEnd() end");
    if (round >= MAX_ROUNDS) {
        onSessionEnd();
    } else {
        onRoundStart();
    }
} <- #200-212 protected void onRoundEnd()
```

on round end code

Saved: 11/25/2025 9:48:02 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup, end checks, and next lifecycle events)

Your Response:

Its trigered when a round finishes(when the word is fully guessed,or max strikes reached or when the last player finishes their turn)

Saved: 11/25/2025 9:48:02 PM

Task #7 (0.57 pts.) - GameRoom Session End

Progress: 100%

Details:

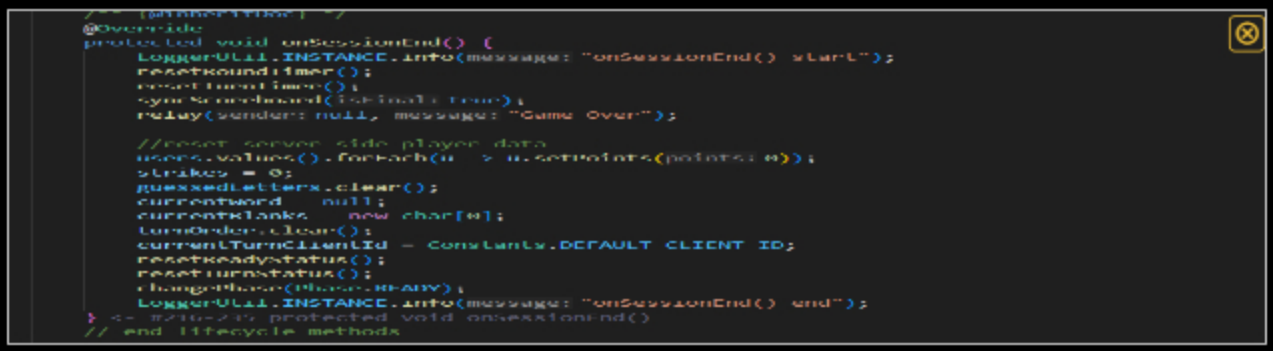
- Reqs from Document
  - **Condition 1:** Session ends when X rounds have passed
  - Send the final scoreboard to all clients sorted by highest points to lowest (include a game over message)
  - Reset the player data for each client server-side and client-side (do not disconnect them or move them to the lobby)
  - A new ready check will be required to start a new session

Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionEnd()`



```
/** {@inheritDoc} */
@Override
protected void onSessionEnd() {
    LoggerUtil.INSTANCE.info(message: "onSessionEnd() start");
    resetRoundTimer();
    resetTurnTimer();
    syncScoreboard(isFinal: true);
    relay(sender: null, message: "Game Over");

    //reset server side player data
    users.values().forEach(u -> u.setPoints(points: 0));
    strikes = 0;
    guessedLetters.clear();
    currentWord = null;
    currentBlanks = new char[0];
    turnOrder.clear();
    currentTurnClientId = Constants.DEFAULT_CLIENT_ID;
    resetReadyStatus();
    resetTurnStatus();
    changePhase(Phase.READY);
    LoggerUtil.INSTANCE.info(message: "onSessionEnd() end");
} // #210-21: protected void onSessionEnd()
// end lifecycle methods
```

on session end code

Saved: 11/25/2025 9:48:54 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

Your Response:

it fully resets the game so players can start a new session with a new ready check

Saved: 11/25/2025 9:48:54 PM

Section #3: (4 pts.) Gameroom User Action And State

Progress: 100%

Task #1 (1.50 pts.) - Word Logic

Progress: 100%

Details:

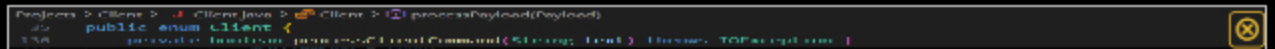
- Reqs from document
  - Command: `/guess <word>`
    - GameRoom will check against the correct word
      - If it matches (exactly)
        - The guesser will receive points for each missing letter and bonus points for solving (i.e., 2x points for each empty space)
          - Points will be stored on the Player/User object
          - Sync the points value of the Player to all Clients
        - A message will be relayed that "X guessed the correct word {word} and got {points} points".
        - The round should end
      - If no matches
        - A strike will be incurred and synced to all Clients
        - A message will be relayed that "X guessed {word} and it was wrong"
        - Their turn should end

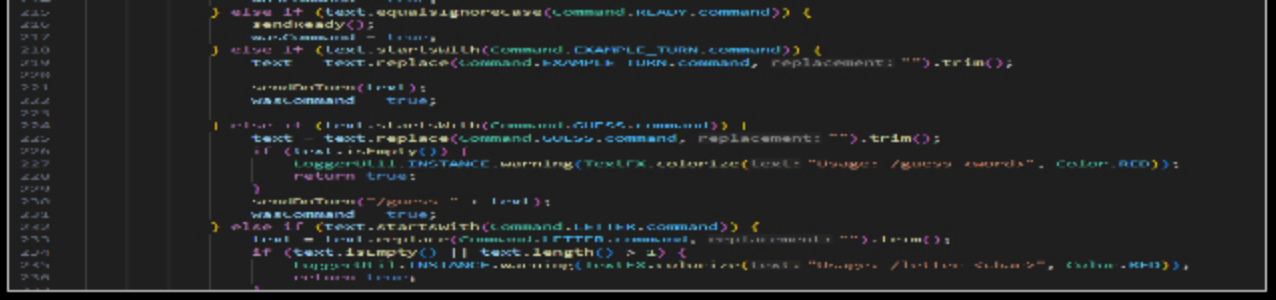
Part 1:

Progress: 100%

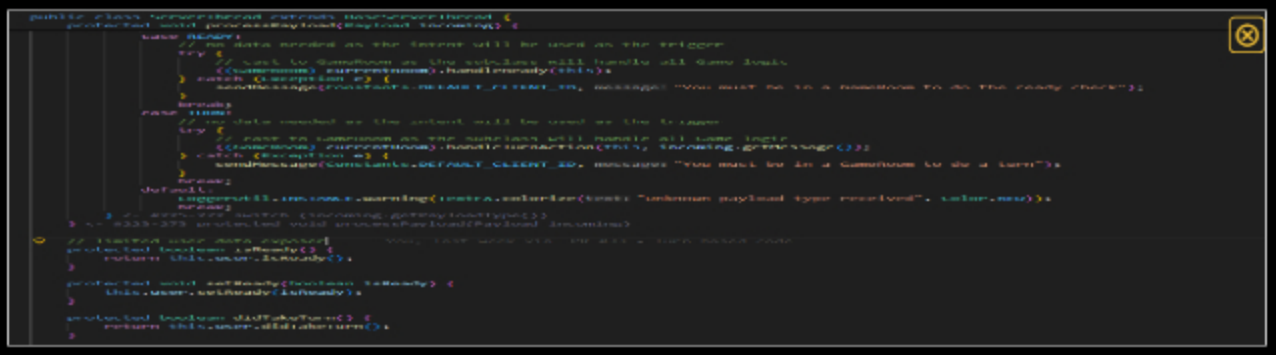
Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

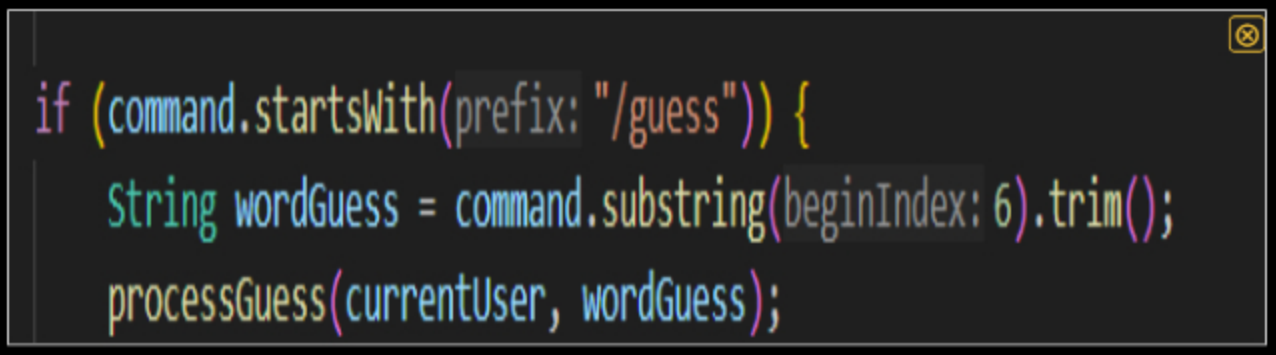




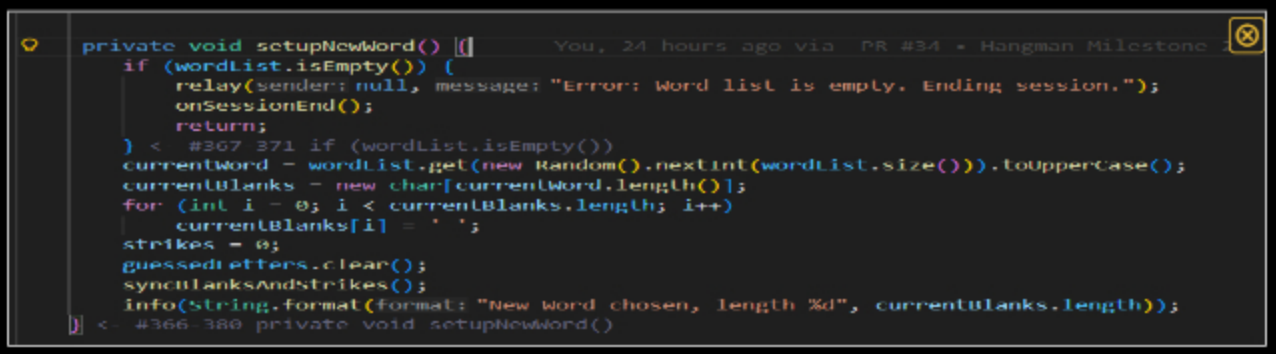
client processing



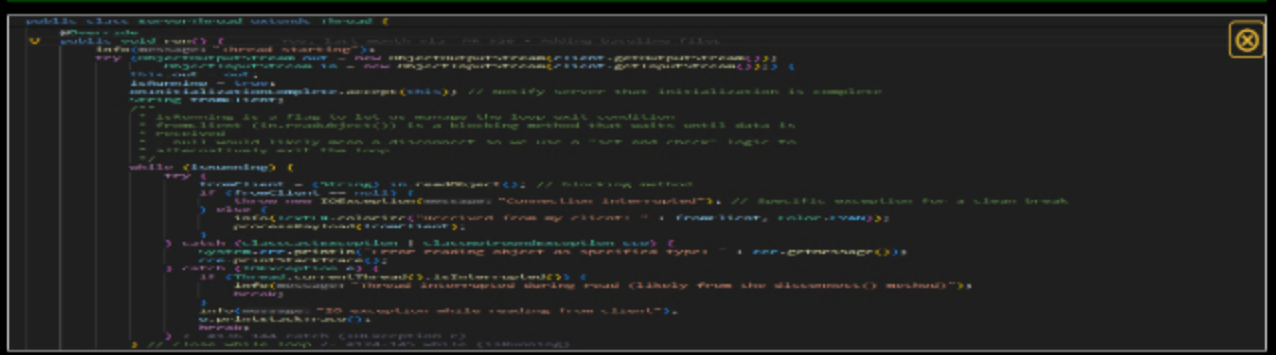
server thread processing



gameroom handling



send/sync



server thread receiving

```
private void listenToServer() {
    try {
        while (isRunning && !isConnected()) {
            Payload fromServer = (Payload) in.readObject(); // blocking read
            if (fromServer != null) {
                processPayload(fromServer);

            } else {
                LoggerUtil.INSTANCE.info("Server disconnected");
                break;
            }
        } // end while (isRunning && isConnected())
    } catch (ClassCastException | ClassNotFoundException cce) {
        LoggerUtil.INSTANCE.severe("Error reading object as specified type:", cce);
        // e.printStackTrace();
    } catch (IOException e) {
        if (isRunning) {
            LoggerUtil.INSTANCE.warning("Connection dropped");
            e.printStackTrace();
        }
    } catch (Exception e) {
        LoggerUtil.INSTANCE.severe("Unexpected error in listenToServer()", e);
    } finally {
        closeServerConnection();
    }
    LoggerUtil.INSTANCE.info("listenToServer thread stopped");
} // end private void listenToServer()
```

client side receiving

```
    case TURN:
        // no data needed as the intent will be used as the trigger
        try {
            // cast to GameRoom as the subclass will handle all Game logic
            ((GameRoom) currentRoom).handleTurnAction(this, incoming.getMessage());
        } catch (Exception e) {
            sendMessage(Constants.DEFAULT_CLIENT_ID, "You must be in a GameRoom to do a turn");
        }
        break;
    default:
        LoggerUtil.INSTANCE.warning(TextFX.colorize("Unknown payload type received", Color.RED));
        break;
} // end switch (incoming.getPayloadType())
} // end protected void processPayload(Payload incoming)
```

server thread processing

Saved: 11/25/2025 10:58:47 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

Client sends TURN payload to ServerThread. it receives it to GameRoom then handles guess. GameRoom calculates result to ServerThread broadcasts updated game state, points, turn info then Clients receive payloads and update display.

Saved: 11/25/2025 10:58:47 PM

Task #2 (1.50 pts.) - Letter Logic

Progress: 100%

Details:

- Reqs from document

- Command: `/letter <letter>`
  - GameRoom will check the occurrences of the letter that haven't been guessed this round
    - If any matches
      - The guesser will receive points for each slot the letter appears in if it has not been previously guessed for this word
        - I.e., points times the number of matched letters
        - Points will be stored on the Player/User object
        - Sync the points value of the Player to all Clients
      - A message will be relayed "X guessed {letter} and there were {number} {letter}'s which yielded {points} points
      - If word is completed, round should end
      - Otherwise, just their turn should end
    - If no matches
      - A strike will be incurred and synced to all Clients
      - A message will be relayed "X guess {letter} letter, which isn't in the word"
      - If strikes reach the limit the round ends
      - If strikes don't reach the limit the current Player's turn ends

Part 1:

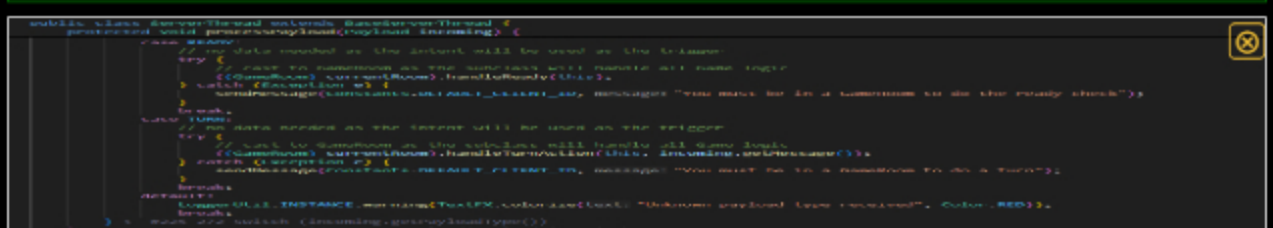
Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)



Client processing





ServerThread processing

```
processGuess(currentUser, wordGuess);
} else if (command.startsWith(prefix: "/letter")) {
    String letterGuess = command.substring(beginIndex: 7).trim();
    processLetter(currentUser, letterGuess);
```

GameRoom Handling

```
private void processLetter(ServerThread guesserThread, String letterGuess) {
    if (letterGuess == null || letterGuess.length() != 1) {
        guesserThread.sendMessage(Constants.DEFAULT_CLIENT_ID, message: "Usage: /letter <single_char>");
        onTurnEnd();
        return;
    } <- #500 512 if (letterGuess == null || letterGuess.length() != 1)

    char letter = Character.toUpperCase(letterGuess.charAt(index: 0));
    if (guessedLetters.contains(letter)) {
        guesserThread.sendMessage(Constants.DEFAULT_CLIENT_ID, "Letter already guessed: " + letter);
        onTurnEnd();
        return;
    } <- #515-519 if (guessedLetters.contains(letter))

    guessedLetters.add(letter);

    int matches = 0;
    for (int i = 0; i < currentWord.length(); i++) {
        if (currentWord.charAt(i) == letter && currentBlanks[i] == '_') {
            currentBlanks[i] = letter;
            matches++;
        }
    } <- #524-529 for (int i = 0; i < currentWord.length(); i++)
```

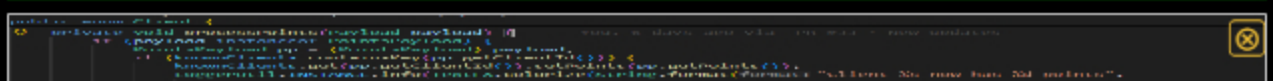
send and sync

```
            break;
        case POINTS:
            processPoints(payload);
            break;
        case GAME_STATE:
            processGameState(payload);
            break;        You, 6 days ago via  PR #33 • New updates
        default:
            LoggerUtil.INSTANCE.warning(TextFX.colorize(text: "Unhandled payload type", Color.YELLOW));
            break;
```

send and sync

```
        }
        break;
    case TURN:
        // no data needed as the intent will be used as the trigger
        try {
            // cast to GameRoom as the subclass will handle all Game logic
            ((GameRoom) currentRoom).handleTurnAction(this, incoming.getMessage());
        } catch (Exception e) {
            sendMessage(Constants.DEFAULT_CLIENT_ID, message: "You must be in a GameRoom to do a turn");
        }
        break;
```

server thread receiving





client receiving

Saved: 11/25/2025 11:49:24 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

Client sends TURN payload to ServerThread. it receives it to GameRoom then handles guess. GameRoom calculates result to ServerThread broadcasts updated game state, points, turn info then Clients receive payloads and update display.

Saved: 11/25/2025 11:49:24 PM

Task #3 (0.50 pts.) - Skip Logic

Progress: 100%

Details:

- Reqs from document
  - Command: `/skip`
    - Skips the player's turn if it's their turn (ends their turn)

Part 1:

Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)

- Show the Client receiving this data (process method)

```
                wasCommand = true;
            } else if (text.equalsIgnoreCase(Command.SKIP.command)) {
                sendDoTurn(text: "/skip");
                wasCommand = true;
            }
        } <- #138-244 if (text.startsWith(Constants.COMMAND_TRIGGER))

        return wasCommand;
    }  <- #136-247 private boolean processClientCommand(String text) throws IOEx...
```

Client processing



ServerThread processing

```
} else if (command.equals(anObject: "/skip")) {
    processSkip(currentUser);
} else {
```

GameRoom Handling

```
        }
        break;
    case TURN:
        // no data needed as the intent will be used as the trigger
        try {
            // cast to GameRoom as the subclass will handle all Game logic
            ((GameRoom) currentRoom).handleTurnAction(this, incoming.getMessage());
        } catch (Exception e) {
            sendMessage(Constants.DEFAULT_CLIENT_ID, message: "You must be in a GameRoom to do a turn");
        }
        break;
```

server thread receiving

```
private void processTurn(Payload payload) {
    // NOTE: For now assuming ReadyPayload (this may be changed later)
    if (!(payload instanceof ReadyPayload)) {
        error(message: "Invalid payload subclass for processTurn");
        return;
    }
    ReadyPayload rp = (ReadyPayload) payload;
    if (!knownClients.containsKey(rp.getClientId())) {
        LoggerUtil.INSTANCE.severe(string.format(format: "Received turn status for client id %s who is not known",
                rp.getClientId()));
        return;
    } // END: !(!knownClients.containsKey(rp.getClientId()))
    User cp = knownClients.get(rp.getClientId());
    cp.setLostTurn(rp.isReady());
```

```
        String message = String.format("%s %s their turn", cp.getDisplayName(),
                cp.didTakeTurn() ? "took" : "reset");
        loggerUtil.INSTANCE.info(message);
    }
} <- www-sw private void processTurn(Payload payload)

private void processPhase(Payload payload) {
    currentPhase = Enum.valueOf(enumClass: Phase.class, payload.getMessage());
    System.out.println(textFx.colorize("current phase is " + currentPhase.name(), Color.YELLOW));
}
```

client receiving

```
protected void handleTurnAction(ServerThread currentUser, String exampleText) {
    // Check if the client is in the room
    try {
        checkPlayerInRoom(currentUser);
        checkCurrentPhase(currentUser, Phase.IN_ROUND);
        checkIsCurrentPlayer(currentUser.getClientId());
        checkIsReady(currentUser);
        if (currentUser.didTakeTurn()) {
            currentUser.sendMessage(Constants.DEFAULT_CLIENT_ID, message: "You have already taken your turn this round");
            return;
        }
        currentUser.setTookTurn(tookTurn: true);
        sendTurnStatus(currentUser, currentUser.didTakeTurn());

        if (exampleText == null || exampleText.isBlank()) {
            currentUser.sendMessage(Constants.DEFAULT_CLIENT_ID, message: "No command provided.");
            onTurnEnd();
            return;
        } // end: else if (exampleText == null || exampleText.isBlank())
        String command = exampleText.toLowerCase().trim();

        if (command.startsWith(prefix: "/guess")) {
            String wordGuess = command.substring(beginIndex: 6).trim();
            processGuess(currentUser, wordGuess);
        } else if (command.startsWith(prefix: "/letter")) {
            String letterGuess = command.substring(beginIndex: 7).trim();
            processLetter(currentUser, letterGuess);
        } else if (command.equals(anObject: "/skip")) {
            processSkip(currentUser);
        } else {
            currentUser.sendMessage(Constants.DEFAULT_CLIENT_ID,
                    message: "Unknown game command. Use /guess <word>, /letter <char>, or /skip.");
            onTurnEnd();
        } // end: else
```

sync and send

Saved: 11/25/2025 11:41:00 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

the client sends skip, server threads reroute it, gameroom marks the sends skip to everyone

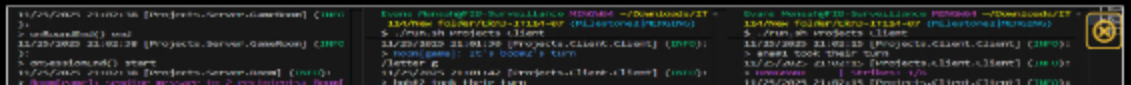
Saved: 11/25/2025 11:41:00 PM

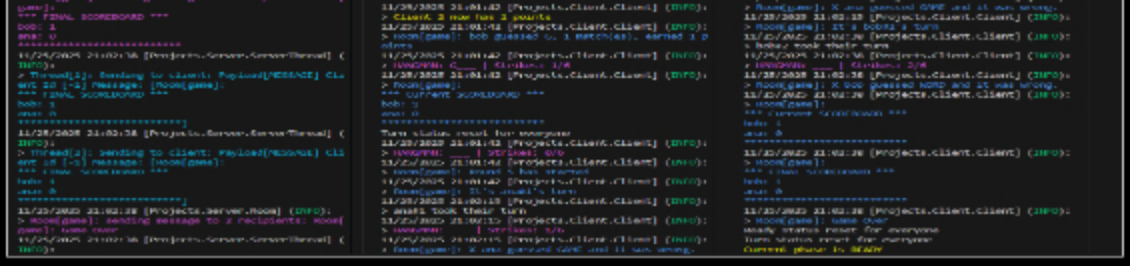
Task #4 (0.50 pts.) - Game Cycle Demo

Progress: 100%

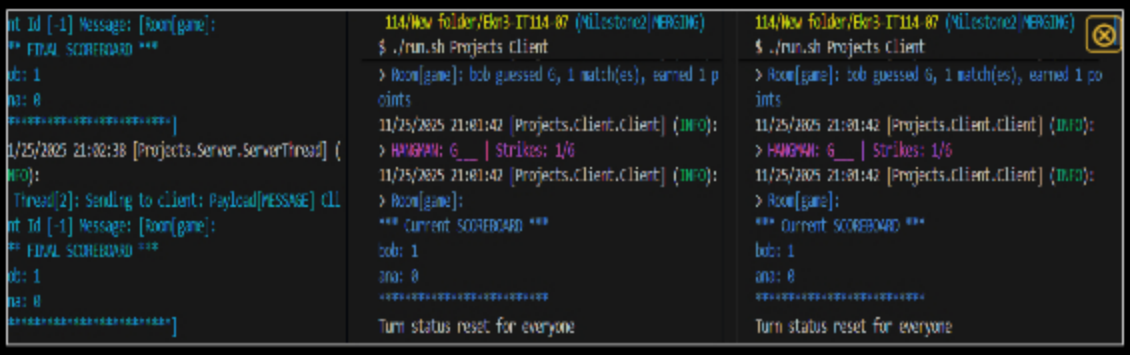
Details:

- Show examples from the terminal of a full session demonstrating each command and progress output
- This includes showing blanks, correct letters, incorrect letters, correct words, incorrect words, skips, scores, and scoreboards, etc
- Ensure at least 3 Clients and the Server are shown
- Clearly caption screenshots

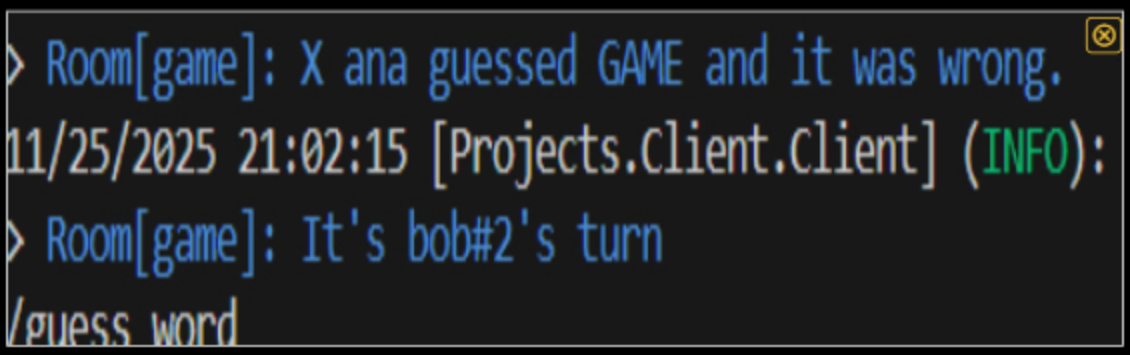




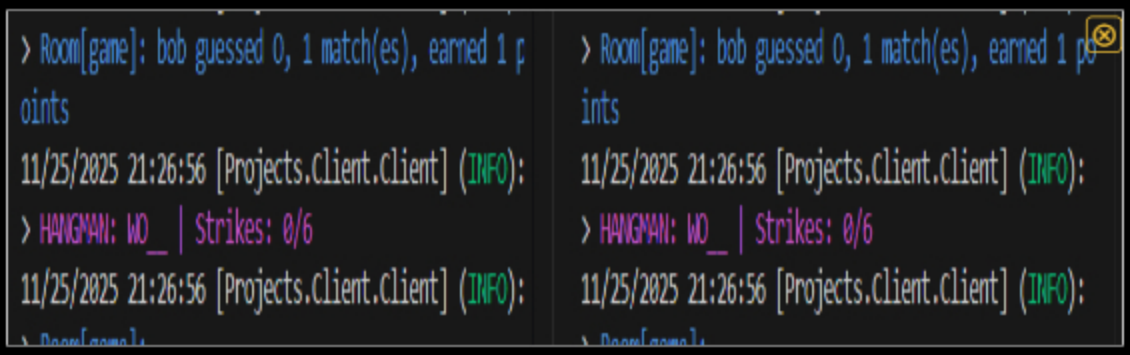
correct letters, incorrect letters and incorrect words



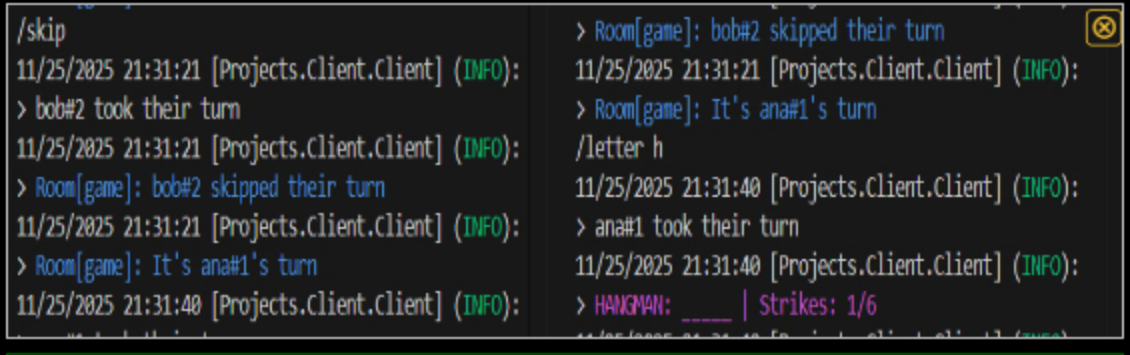
scoreboard



incorrect word



correct words



skipped turn

Saved: 11/25/2025 9:37:46 PM

Section #4: (1 pt.) Misc

Progress: 100%

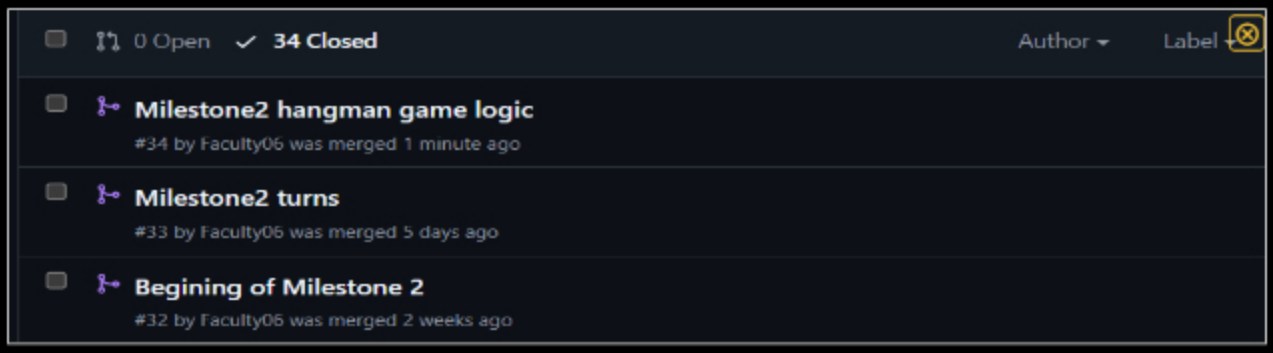
Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



pull request

Saved: 11/25/2025 3:24:31 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/Faculty06/Ekn3-
IT1

URL
https://github.com/Faculty06/Ekn

Saved: 11/25/2025 3:24:31 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waka Time



wakatime



waka time

Saved: 11/25/2025 3:12:43 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

how to sync points across the server and client

Saved: 11/25/2025 3:07:15 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

overriding to string

Saved: 11/25/2025 3:06:25 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Managing the game logic and syncing everything to client

Saved: 11/25/2025 3:05:14 PM